

RAG : Retrieval - Augmented Generation

What is RAG?

info retrieval @ LLMs' generative ability

from ext sources
organiz^o internal knowledge
no need to retrain model

provided to models
as context

⇒ model generates answers that are more

⇒ reduces hallucinations

⇒ ↑ trust / reliability / transparency

up-to-date
accurate
factual
grounded in real data
↳ ≠ only from memo

why is RAG needed? trained on fixed data ⇒ hallucinat^o
↳ general data ⇒ not spc^l ⇒ fine-tune ⇒ expensive ++

RAG PIPELINE - from query to answer

QUERY TRANSLATION

Transf query into enhanced forms before the search, two ways:

- * query decomp into subqueries → RAG-Fusion
- * pseudo-doc : generate response & use it to search similar docs.
↳ embedded ↳ hallucin^o

ROUTING

Decide in which subsets of DB search, based on the query two ways:

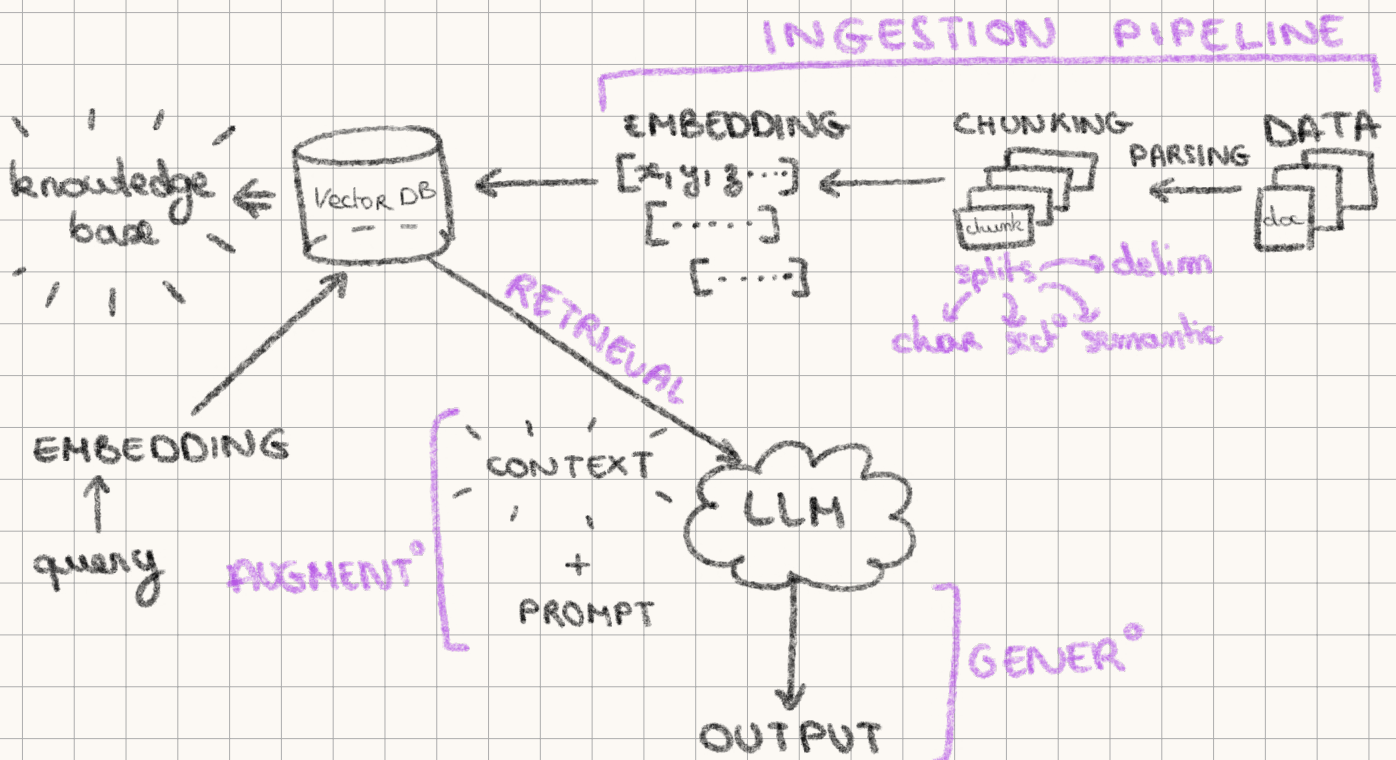
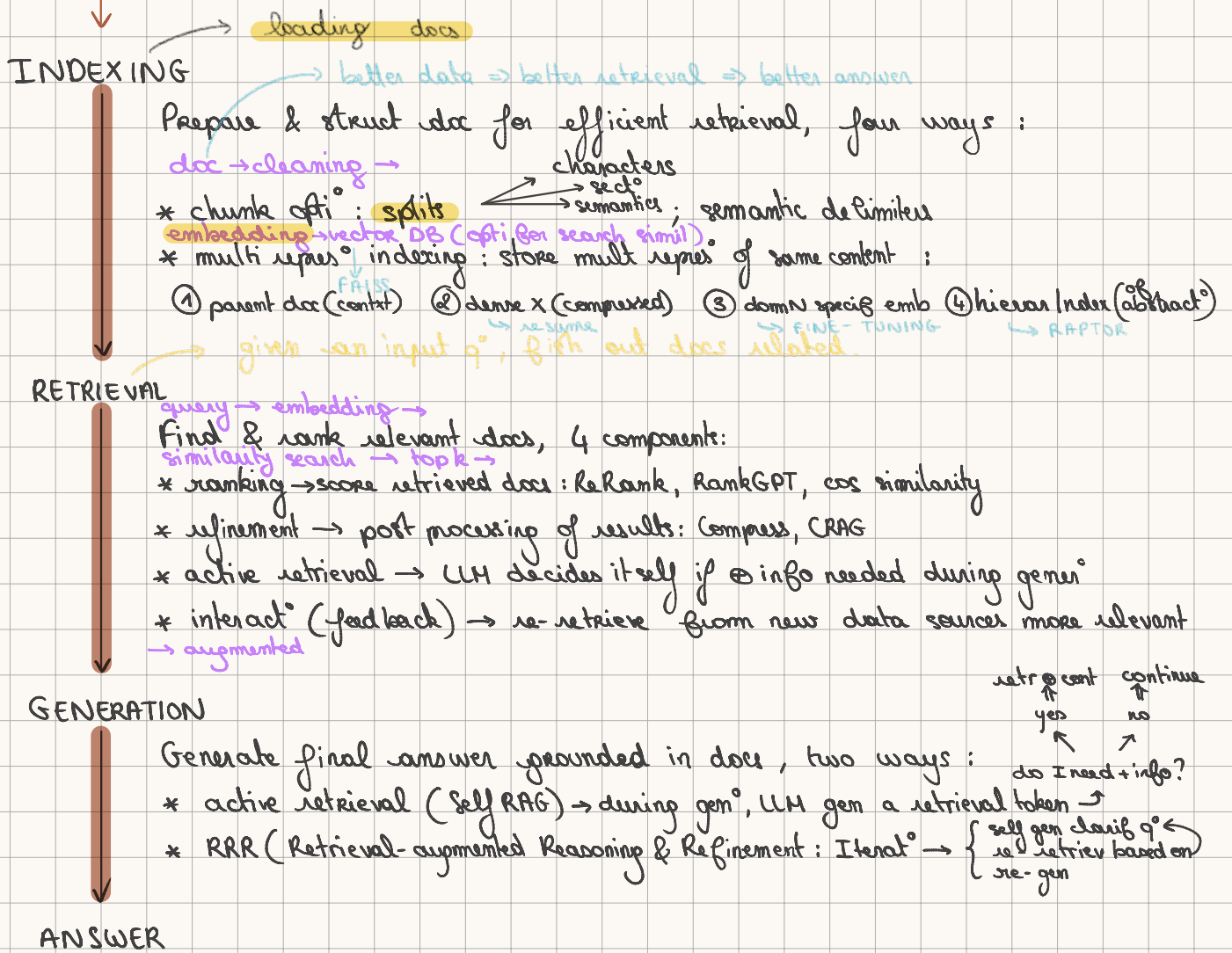
- * logical : based on keywords.
- * semantic : emb the query & compare it to several prompts.

QUERY CONSTRUCTION

Decide which type of DB query, 3 sub-types:
↳ dep on the nature of the query

- * text to SQL (Relational DBs): convert natural lang q^o into SQL query
- * text to cypher (Graph DBs): convert into cypher (= query for graph)
- * self query retriever (Vector DBs): LLM autogener filtering metadata from query

RAG PIPELINE :

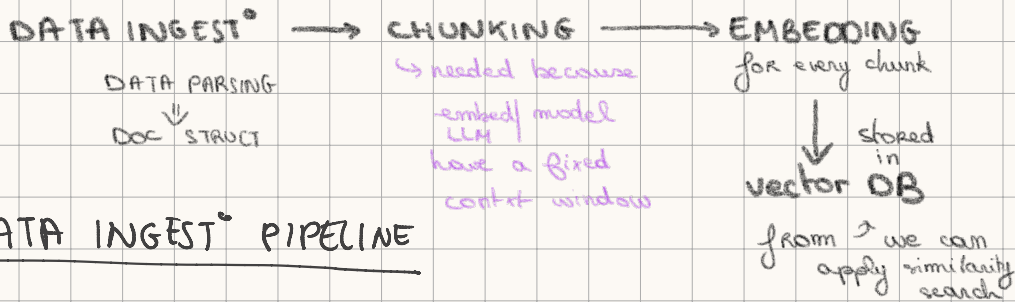


1) INGESTION TO VECTOR DB PIPELINE

↳ to know about data struct

↳ convert DATA (in any formats) to struct
complete RAG pipeline :

- Data Ingest° Pipeline
- Query Retrieval Pipeline



I / DATA INGEST° PIPELINE

① Langchain Doc struct

Doc struct =

- page content (str) → contains the content of a partic file (details)
- metadata (dict) → additional info about (name, nb of pages ...)

useful bc:
once we store into vect DB, when doing simi-search
we can apply filters on it

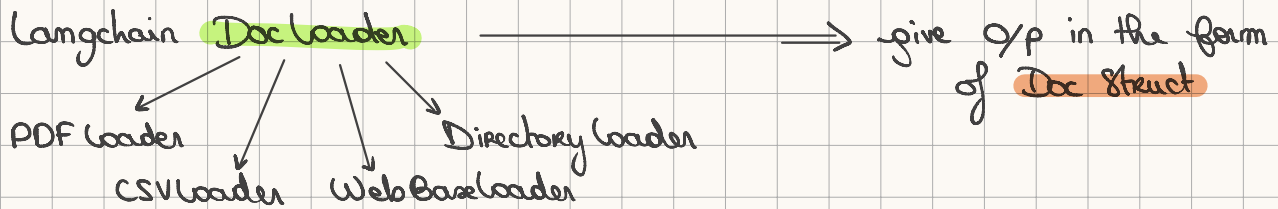
```
doc = Document(  
    page_content = "this is the main content I am using to create RAG",  
    metadata = {  
        "source": "example.txt",  
        "pages": 1,  
        "author": "Mathura Santhalingam",  
        "date_created": "20/08/2026"  
    }  
)  
doc
```

★ chain (minimal vers°): prompt template | model | parser

↳ RAG chain : q° → doc search → injected in prompt → answer generat°

dependencies list (requirement.txt) :

- ↳ langchain: ppl framework (chains + agents + logic) → pre-built struct of reusable code / tools → provides foundation & rules
- ↳ langchain-core: core found° with base abstract°, interfaces, LCEL
- ↳ langchain-community: doc loaders, vector stores, API connectors ---
- ↳ pypdf: python lib used to read, split, extract text from PDF files



```

### Directory Loader(how to read the text above using DirectoryLoader, i want to read the text when i have all the important files in my directory)
from langchain_community.document_loaders import DirectoryLoader

# load all the text files from the directory
dir_loader = DirectoryLoader(
    "../data/text_files",
    glob="**/*.txt", ## pattern to match files on the files
    loader_cls= TextLoader, ##loader class to use, which file we plan to load, if pdf, write pdf instead of TextLoaders
    loader_kwargs={'encoding': 'utf-8'},
    show_progress=False
)

documents=dir_loader.load()
documents
  
```

> https://python.langchain.com/docs/integrations/document_loaders/

② Chunking

> emb model available in sentence transf

> uuid = universal unique id → bc every record that we insert into the vect DB will have an id

> class EmbManager = resp of emb
↳ every class needs an init go

```

class EmbeddingManager:
    """Handles document embedding generation using SentenceTransformer"""

    def __init__(self, model_name: str = "all-MiniLM-L6-v2"):
        """
        Initialize the embedding manager

        Args :
            model_name : HuggingFace model name for sentence embeddings
        """
        self.model_name = model_name
        self.model = None
        self.load_model()
  
```

(protected) internal usage → defined just below
→ empty attribut
→ method to actually load the model
→ const → pass → by default

Vector Storage = Vect DB

```

class VectorStore:
    """Manages document embedding in a ChromaDB vector store"""

    def __init__(self, collection_name: str = "pdf_documents", persist_directory: str = "../data/vector_store"):
        """
        Initialize the vector store

        Args:
            collection_name: Name of the ChromaDB collection
            persist_directory: Directory to persist the vector store
        """
        self.collection_name = collection_name
        self.persist_directory = persist_directory
        self.client = None
        self.collection = None
        self._initialize_store()

    def _initialize_store(self):
        """Initialize ChromaDB client and collection"""
        try:
            # Create persistent ChromaDB client
            os.makedirs(self.persist_directory, exist_ok=True)
            self.client = chromadb.PersistentClient(path=self.persist_directory)

            # Get or create collection
            self.collection = self.client.get_or_create_collection(
                name=self.collection_name,
                metadata={"description": "PDF document embeddings for RAG"}
            )
            print(f"Vector store initialized. Collection: {self.collection_name}")
            print(f"Existing documents in collection: {self.collection.count()}")

        except Exception as e:
            print(f"Error initializing vector store: {e}")
            raise
  
```

create clt which will have a reference to the chromaDB vector store

→ create coll

↳ where we are going to store the vectors in the vectorstore

whatever doc I get,
I zip it (type + with emb)
then mind = acc id
two info needed → emb = np.ndarray ← gener emb
docs list

retriever is build on the top
of a vector DB
→ interface based on a query
& is gonna give you the
response back

retrieve^q = retrieve based
on a query.

```
class RAGRetriever:
    """Handles query-baser retrieval from the vector store"""

    def __init__(self, vector_store: VectorStore, embedding_manager: EmbeddingManager):
        """
        Initialize the retriever

        Args:
            vector_store: Vector store containing document embeddings
            embedding_manager: Manager for generating query embeddings
        """

        self.vector_store = vector_store
        self.embedding_manager = embedding_manager
```

whenever we get query

↓
emb
to make
simi search
in retriever

```
def retrieve(self, query: str, top_k: int = 5, score_threshold: float = 0.0) -> List[Dict[str, Any]]:
    """
    Retrieve relevant documents for a query

    Args:
        query: The search query
        top_k: Number of top results to return
        score_threshold: Minimum similarity score threshold

    Returns:
        List of dictionaries containing retrieved documents and metadata
    """

    print(f"Retrieving documents for query: '{query}'")
    print(f"Top K: {top_k}, Score threshold: {score_threshold}")

    # Generate query embedding
    query_embedding = self.embedding_manager.generate_embeddings([query])[0]

    # Search in vector store
    try:
        results = self.vector_store.collection.query(
            query_embeddings=query_embedding.tolist(),
            n_results=top_k
        )

        # Process results
        retrieved_docs = []
```

the vect store coll, query (- -)

once we receive results ↑
there will be a key called document^s
which we add
in context (ret-c)

```
if results['documents'] and results['documents'][0]:
    documents = results['documents'][0]
    metadatas = results['metadatas'][0]
    distances = results['distances'][0]
    ids = results['ids'][0]

    # we zip them and compare distance
    for i, (doc_id, document, metadata, distance) in enumerate(zip(ids, documents, metadatas, distances)):
        # Convert distance to similarity score (ChromaDB uses cosine distance)
        similarity_score = 1 - distance

        if similarity_score >= score_threshold:
            # we add it inside the context documents which is created in retrieved_docs
            retrieved_docs.append({
                'id': doc_id,
                'content': document,
                'metadata': metadata,
                'similarity_score': similarity_score,
                'distance': distance,
                'rank': i + 1
            })

    print(f"Retrieved {len(retrieved_docs)} documents (after filtering)")
else:
    print("No documents found")
    return retrieved_docs

except Exception as e:
    print(f"Error during retrieval: {e}")
    return []

rag_retriever = RAGRetriever(vector_store, embedding_manager)
rag_retriever
```

Ce qu'il s'est passé :

1. La base a trouvé les 5 meilleurs morceaux parmi les 71.
2. Le 1er avait un score de 0.158 . IPYNB
3. Le 4ème avait un score de 0.005 (très limite !). IPYNB
4. Le 5ème avait un score négatif (il n'avait vraiment *aucun* rapport avec la requête)
5. Ton code a donc ignoré ce 5ème résultat grâce à ta condition de filtrage.

```
# example with a query
rag_retriever.retrieve("What is attention is a chain of verification")
✓ 0.3s
```

```
Retrieving documents for query: 'What is attention is a chain of verification'
Top K: 5, Score threshold: 0.0
Generating embeddings for 1 texts...
Batches: 100%|██████████| 1/1 [00:00<00:00, 3.02it/s]
Generated embeddings with shape: (1, 384)
Retrieved 4 documents (after filtering)
```

II / QUERY RETRIEVAL PIPELINE + LLM METHOD

→ help us gen summarized output in the RAG

RAG → Augmented gen^o

